SubstiChoose — Hackathon MVP Plan

Oct 9, 2026 · @Johndel

Overview

SubstiChoose is an offline Android app: snap a recipe, get its ingredient list, and get safe substitutes for anything you're allergic to or don't have. Gemma 4 E2B runs entirely on the phone, so no recipe or health data leaves the device.

Who it's for

- People with allergies or intolerances who need to adapt a recipe safely (nuts, dairy, gluten, egg, shellfish, soy).
- Home cooks missing an ingredient who need to know what they can use instead.

MVP scope (1 day)

Decision	Choice
Input	Photo of a recipe from a cookbook page or a screen
Platform	Android, Kotlin, Jetpack Compose
AI	Gemma 4 E2B, on-device, offline
Substitution reasons	Allergy/intolerance and "I don't have it"
Storage	Local only (Room); no accounts, no cloud
Output	Ingredient list with allergens flagged, and substitutes per ingredient

Out of scope for the MVP: barcode/label scanning, dish-photo recognition, iOS, accounts and sync, nutrition data.

User flow and screens

The demo flow is five taps: set allergies once, scan, check the list, tap an ingredient, save the adapted recipe.

1. **Onboarding / My Allergies** (first launch only): pick from chips (peanut, tree nuts, dairy, egg, gluten, soy, shellfish, fish, sesame) and add custom ones. These are stored and sent with every AI request.

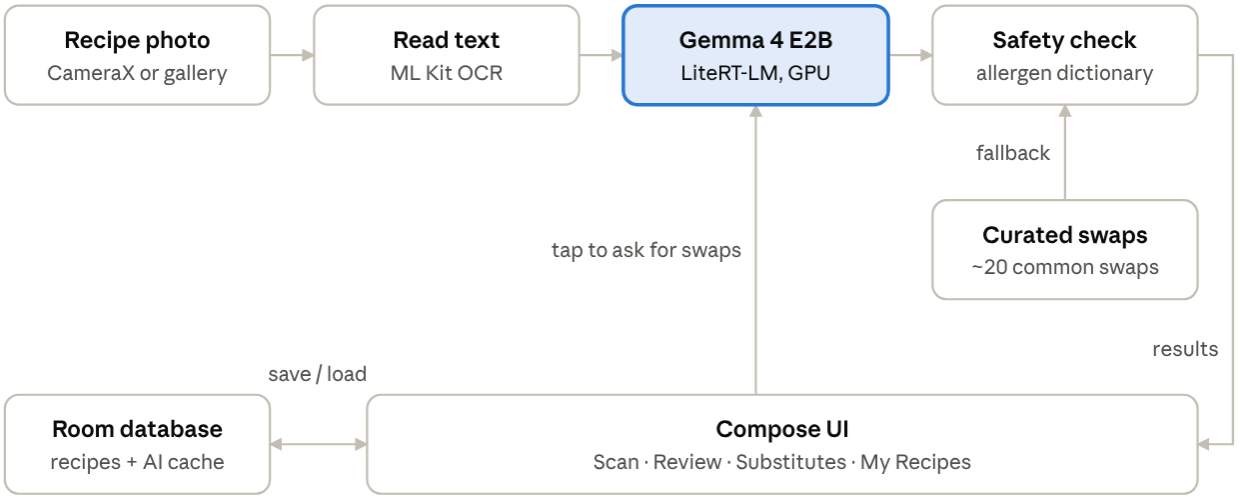
2. **Home — My Recipes**: a list of saved recipes with a warning badge (e.g. "2 allergens"), search, and a big **Scan recipe** button.
3. **Scan**: CameraX preview with a capture button and a **Pick from gallery** option. Show a "Reading recipe…" state while the image is processed.
4. **Review Ingredients**: the extracted list as editable rows (quantity, unit, name). Allergen rows are flagged red automatically. Each row has a toggle: **Don't have it**. The user can fix OCR mistakes, add or delete rows here.
5. **Ingredient Detail / Substitutes**: tap a flagged or toggled ingredient to see 2–3 substitutes as cards: name, ratio ("1 egg = 1 tbsp ground flax + 3 tbsp water"), effect on taste/texture, and a safety note. Pick one.
6. **Adapted Recipe**: the full recipe with substitutes swapped in, plus AI-adjusted steps if the stretch goal is done (e.g. "rest the flax mix 5 minutes before adding"). **Save** stores it in My Recipes.
7. **Shopping list** (stretch): substitutes the user still needs to buy.

One design rule throughout: AI suggestions for allergies always show a short disclaimer ("Check labels — AI suggestions may be wrong").

Architecture and tech stack

A plain single-module Kotlin app: ML Kit reads the text, Gemma 4 E2B understands it, and a Kotlin allergen check has the final say. Nothing needs the internet after the model is on the phone.

Every step runs on the phone, so the app works offline



SubstiChoose architecture · all on-device

The photo becomes text, Gemma turns it into structured ingredients, and every result passes the safety check before the UI shows it.

Layer	Choice	Why
UI	Kotlin, Jetpack Compose, Material 3	Fastest way to build modern Android screens
Camera	CameraX	Simple capture with a stable API
OCR	ML Kit Text Recognition (on-device)	Free, offline, reliable on printed text
LLM runtime	LiteRT-LM (Google AI Edge), Kotlin API	Google's on-device runtime for Gemma 4; CPU, GPU and NPU backends
Model	Gemma 4 E2B instruction-tuned, LiteRT-LM build (about 2.58 GB)	Small enough for phones; reads text, images and audio
Storage	Room + DataStore	Local recipes and settings
JSON	kotlinx.serialization	Parses the model's replies
Async	Coroutines + Flow; simple manual DI (skip Hilt)	Less setup during a hackathon

Why ML Kit before Gemma, not the photo straight to Gemma: Gemma 4 E2B can read images on its own, but dedicated OCR is faster and more dependable on dense recipe text. Gemma then does only the understanding. Keep Gemma's vision as a second try for messy photos — it also makes a good demo moment.

Speed to expect: Google reports about 52 tokens/second decoding for Gemma 4 E2B on an Android GPU. A substitutes reply of about 150 tokens should take roughly 3 seconds. Stream it into the UI so it feels faster.

Getting started: install Google AI Edge Gallery on the demo phone and try Gemma 4 E2B there before writing any code. Then follow the LiteRT-LM Android guide linked from Google's announcement for the current Gradle dependency.

AI design

Gemma gets three small, focused jobs, each returning strict JSON. Allergen safety is checked by plain Kotlin code, never by the model alone.

Job 1 — Extract ingredients. Input: OCR text of the recipe (or the photo itself if you use Gemma's vision). Output:

```
{"title": "Banana Bread",
 "ingredients": [{"qty": "2", "unit": "pcs", "name": "egg", "raw": "2 large
eggs"}],
 "steps": ["Preheat oven to 175°C", "..."]}
```

Job 2 — Suggest substitutes (one call per ingredient, on tap, so it feels fast). Input: the ingredient, the recipe title, the reason (allergy or don't have), and the user's full allergy list. Output: 2–3 options with `name`, `ratio`, `effect`, `note`.

Job 3 — Adapt the steps (stretch goal). Input: the original steps and the chosen swaps. Output: the rewritten steps, with changed lines marked.

Prompt rules

- System prompt: "You are a cooking assistant. Reply with JSON only, matching this schema: …" Include one short example.
- Keep temperature low (about 0.2) for extraction, a little higher (0.5) for substitutes.
- Parse with kotlinx.serialization. On a parse failure, retry once with "Your last reply was not valid JSON"; then fall back to showing the raw text.

Safety guardrails

- A local **allergen dictionary** (JSON in assets) maps ingredient words to allergens: butter, ghee, whey → dairy; flour, soy sauce → gluten; patis, bagoong → fish/shellfish. It flags rows on the Review screen without the AI.
- Every substitute the model returns is run through the same dictionary. Anything that hits the user's allergies is dropped (e.g. almond milk for a nut allergy).
- A small **curated substitutes table** (about 20 common swaps: egg, butter, milk, flour, buttermilk…) answers instantly and is the fallback if the model is slow or fails. Good for the live demo too.

Data model

Four Room tables cover the MVP. Allergies live in DataStore because there is only one user.

Table	Key fields	Notes
`Recipe`	id, title, imagePath, stepsJson, adaptedStepsJson, createdAt	One row per scanned recipe
`Ingredient`	id, recipeId, qty, unit, name, raw, isAllergen, isMissing	`isAllergen` set by the dictionary; `isMissing` is the "Don't have it" toggle

Table	Key fields	Notes
`Substitute`	id, ingredientId, name, ratio, effect, note, source, isChosen	`source` = AI or curated
`SubCache`	ingredientName + allergyHash, json	Stretch goal: caches AI answers so repeat lookups are instant
`Knowledge`	id, dishName, ingredients, allergens, notes, learnedSwapsJson, createdAt	Stretch goal: what users teach the app; FTS4 search, top 3 matches go into the prompt

DataStore: `allergies: Set<String>`, `onboardingDone: Boolean`, `modelDownloaded: Boolean`.

1-day build schedule

This schedule assumes about 10–12 working hours. Each person owns one layer and codes against the shared JSON schemas from the first hour, so the parts plug together at integration. Get Gemma answering on the demo phone first: it has the most unknowns.

Person	Role	Owns
P1	AI lead	LiteRT-LM setup, loading Gemma, the three prompts, JSON parsing
P2	Camera and data	CameraX, ML Kit OCR, Room/DataStore, wiring the pipeline
P3	UI	Compose screens, navigation, loading/error/empty states
P4	Safety, content and demo	Allergen dictionary, curated swaps, test recipes, QA, slides and pitch

Hour 0–1 — Set up together (end state: model runs on the phone, everyone has the repo)

- [x] ~~P1: copy the model onto the demo phone and run Gemma 4 E2B in AI Edge Gallery to check speed~~
- [x] ~~P2: create the project in VS Code (Gradle) (Compose, Room, ML Kit, LiteRT-LM) and push it to a shared repo~~
- [x] ~~P3: build Home from the mockup (recipe list + camera button at bottom center) and agree on navigation~~

- [x] ~~P4: write down the JSON schemas from AI design so everyone codes to the same shapes; collect 6 test recipes~~

Hours 1–5 — Build in parallel (end state: each part works on its own)

- [ ] P1: load Gemma from phone storage; extraction prompt works on the test recipes
- [ ] P1: substitutes prompt + JSON parsing with one retry
- [ ] P2: CameraX capture + gallery picker; ML Kit returns text
- [ ] P2: Room tables (Recipe, Ingredient, Substitute) and allergies in DataStore
- [ ] P3: My Allergies, Home, Scan, Review Ingredients and Substitutes screens with fake data
- [x] ~~P4: allergen dictionary JSON (including patis, bagoong), 24 curated swaps, and the Kotlin filter (in the SubstiChoose-P4 folder)~~

Hours 5–8 — Integrate and test (end state: the full flow works on the phone)

- [ ] P1 + P2: wire photo → OCR → Gemma → safety check → screens
- [ ] P3: swap fake data for real; loading states and the disclaimer
- [ ] P4: run every test recipe end to end; log failures for P1 to fix in the prompts
- [ ] All: save flow works; pre-save 2 demo recipes as a backup

Final 2–3 hours — Freeze and rehearse (end state: rehearsed 3-minute demo)

- [ ] All: code freeze 2 hours before judging; after that, fix only demo-breaking bugs
- [ ] P3: polish the demo path only
- [ ] P4: slides + a backup screen recording of one full run
- [ ] All: rehearse the demo twice on the demo phone in airplane mode

Cut to stretch goals for one day: AI-adjusted cooking steps (Job 3), SubCache, Gemma vision fallback, shopping list, Teach SubstiChoose.

Stretch goal: Teach SubstiChoose

The app can learn dishes Gemma doesn't know by saving the user's notes on the phone and adding the best matches to each prompt. The model itself is never retrained. Build this only if the core flow works by hour 6.

1. **Something unknown comes up:** a family recipe or a local dish (e.g. pinangat, kinilaw) where Gemma flags it as unfamiliar or answers weakly.
2. **The user teaches the app:** a short "Teach SubstiChoose" form with the dish name, main ingredients, what it usually contains ("has coconut milk, shrimp paste") and swaps that worked.

3. **It's saved on the phone:** in a `Knowledge` table (see Data model). When a user picks a substitute and saves a recipe, that choice is also stored as a learned swap.
4. **Later questions use the notes:** before calling Gemma, the app searches `Knowledge` and adds the top 3 matches to the prompt. The safety check still runs on every answer.

What gets added to the prompt:

```
Known facts from this user (may be incomplete):
- Pinangat: taro leaves, coconut milk, shrimp paste (bagoong) -> shellfish
- Learned swap: shrimp paste -> salt + mushroom seasoning (worked for this
user)
```

Rules

- Add at most 3 entries, because Gemma can only read a limited amount of text at once.
- User notes can add allergen warnings but never remove one from the built-in allergen dictionary.
- Everything stays on the phone, which fits the offline, private pitch.

One-day build (about 2 hours): keyword search with Room full-text search (FTS4) on dish and ingredient names. P4 builds the Teach form and the table; P1 adds the matches to the prompts.

After the hackathon: search by meaning with a small on-device text-embedding model (e.g. EmbeddingGemma), so "laing" also finds "taro leaves in coconut milk".

Risks, demo and next steps

Risk	Fallback
The model (about 2.6 GB) is too big to ship in the APK	Sideload it with `adb push` onto the demo phone. A live demo is enough, so no APK distribution is needed
Slow or out-of-memory on older phones	Confirm the demo phone has 8 GB+ RAM in the first hour; use the GPU backend; keep prompts short
Messy OCR on handwriting or two-column pages	Review screen lets users fix rows; pass the cropped photo to Gemma vision as a second try
Model invents an unsafe substitute	Dictionary filter + curated table + disclaimer (see AI design)

Risk	Fallback
Live scan fails on stage	Pre-saved demo recipes and a backup video

Demo script (3 minutes)

1. Set "egg" and "dairy" as allergies.
2. Turn on airplane mode to prove it's fully offline.
3. Scan a printed banana bread recipe; eggs and butter light up red.
4. Tap eggs → pick flax egg. Toggle "Don't have it" on baking soda → get baking powder with a ratio.
5. Save it and show it in My Recipes.
6. If the stretch goal is done: scan a dish the app doesn't know, teach it, scan again, and show the answer now uses what you taught.

Stretch goals: shopping list, local Filipino swaps (e.g. calamansi for lemon), pantry tracking, iOS, search by meaning for Teach SubstiChoose.

Open questions

- Which Android phone model is the demo phone, and how much RAM does it have? P1 checks this in the first hour with AI Edge Gallery.

Sources

- Blazing fast on-device GenAI with LiteRT-LM — Google Developers Blog
- Gemma 4 edge deployment: E2B and E4B models — MindStudio
- Gemma 4 E2B LiteRT-LM builds — Hugging Face